

Week 1 Applied Class

Terminal and Basic File Operations Lab

Overview

Week 1 is a terminal setup and first file-operations lab. By the end, you should have a FIT2109 workspace and practice with `pwd`, `ls`, `cd`, `mkdir`, `cp`, and `mv`.

Working rule. Ask for help early if you cannot open a terminal, cannot find your home directory, cannot extract the starter zip, or cannot run `./check_progress.sh`. Those are the important Week 1 outcomes.

Objectives

- To open a supported Unix-like command-line environment.
- To create and enter a FIT2109 workspace under your home directory.
- To download, extract, and inspect the Week 1 starter project.
- To use `pwd` and `ls` to orient yourself.
- To use `cd`, `mkdir`, `cp`, and `mv` for basic file and folder work.
- To run the checker script and interpret its output.
- If time permits, to preview the larger shell-rescue tasks that continue in Week 2.

Task 0: Open a Supported Terminal

Before this class you should already have a working terminal, set up by following the **Week 0 getting started guide** in the unit materials. That guide has full step-by-step instructions for Windows (Ubuntu on WSL 2), macOS, and Linux. If you have not set up yet, or your terminal is not working, follow that guide first and ask on Ed if you get stuck.

Open your terminal.

- **macOS:** press `Cmd + Space`, type `Terminal`, and press `Enter`.
- **Windows:** open `Ubuntu` from the Start menu (or type `wsl` in PowerShell).
- **Linux:** open your terminal emulator (often `Ctrl + Alt + T`).

Once your terminal is open, confirm it works.

Type and run:

```
pwd
echo "$SHELL"
```

What you are checking.

- `pwd` prints a directory path.
- `echo "$SHELL"` prints the shell used by your terminal.

macOS users: your interactive shell may be Zsh. That is fine. Week 1 commands such as `pwd`, `ls`, `cd`, `mkdir`, `cp`, and `mv` behave the same for our purposes.

First Commands You Will Use

The shell is always “standing” in one directory. Most beginner confusion comes from not knowing where the shell is standing before running a command. The first commands below are enough to orient yourself and do simple file operations.

- `pwd` means “print working directory”. It shows the folder where the shell is currently standing, for example `pwd` might print `/home/you/fit2109`.
- `ls` means “list”. It shows files and folders. `ls project` lists the `project` folder instead of the current folder.
- `cd` means “change directory”. It moves the shell to another directory, for example `cd ~/fit2109`.
- `mkdir` means “make directory”. It creates a new folder, for example `mkdir notes` makes a folder called `notes`. The `-p` flag does two helpful things: it can create a whole chain of folders at once (for example `mkdir -p ~/fit2109/week1/data` creates `fit2109`, then `week1` inside it, then `data` inside that, even if none of them exist yet), and it does not show an error if the folder is already there.
- `cp` means “copy”. It creates a second copy of a file, for example `cp a.txt b.txt`. The original file remains where it was.
- `mv` means “move”. It changes a file’s location or name, for example `mv a.txt archive/a.txt`. The old pathname disappears.

Paths in this lab.

- `~` means your home directory.
- `~/fit2109` means the `fit2109` folder inside your home directory.
- `project` is a relative path: it means “the `project` folder from where I am standing now”.

Task 1: Create Your FIT2109 Workspace

Create a folder for FIT2109 work under your home directory:

```
mkdir -p ~/fit2109
cd ~/fit2109
pwd
```

Questions.

1. What path did `pwd` print?
2. In `~/fit2109`, what does the leading `~` mean?
3. Why is it useful to keep weekly applied work under one course folder?

Task 2: Get the Week 1 Starter Project

Download the Week 1 basic commands starter zip from Ed. Extract it, then place the extracted folder under your FIT2109 workspace so that its path is `~/fit2109/week1-rescue`.

Move into the folder.

Type and run:

```
cd ~/fit2109/week1-rescue
```

Check that the folder contains `check_progress.sh` and a `project/` directory.

Type and run:

```
ls
ls project
```

First checker run. `check_progress.sh` is a script included in the starter project. It checks your work for this lab and prints a score out of 3, with a `[PASS]` or `[FAIL]` line for each task, so you can see what is still left to do. The leading `./` tells the shell to run the script from the current folder.

Type and run:

```
./check_progress.sh
```

The first score should be below 100%. That is expected. The remaining tasks will make the checker pass.

Task 3: Make a Folder and Copy a File

Goal. Create an archive folder and copy the team notes into it.

What you must do.

- Create a new folder called `archive` inside the `project` folder (so its path becomes `project/archive`).
- Copy the file `team-notes.txt` (located in the `project/notes` folder) into the new `archive` folder.
- Name the copy `team-notes-copy.txt`.
- Check that both the original and copied file exist.

Commands to use.

```
mkdir project/archive
cp project/notes/team-notes.txt project/archive/team-notes-copy.txt
ls project/notes
ls project/archive
```

Questions.

1. Which command created the archive folder?
2. After `cp`, why should the original notes file still exist?
3. Which two folders did you list to check your work?

Task 4: Rename a File

Goal. Rename an old temporary file so it is clearly marked as archived.

What you must do.

- Rename `project/tmp/old-debug.txt` to `project/tmp/old-debug.archived`.
- Check that the old name is gone and the new name exists.

Note on the new name. Dropping the `.txt` ending here is intentional. A file name ending such as `.txt` is only a label that hints at the file's type; it does not change the contents, and `mv` lets you change it freely. The `.archived` ending is used as a simple convention to mark the file as old and no longer in active use. The checker expects exactly `old-debug.archived` with no `.txt`.

Commands to use.

```
mv project/tmp/old-debug.txt project/tmp/old-debug.archived
ls project/tmp
```

Questions.

1. What changed after running `mv`?
2. Why is `mv` different from `cp`?
3. Which command showed you the current contents of `project/tmp`?

Checkpoint. Type and run:

```
./check_progress.sh
```

Your goal for Week 1 is to reach **3/3**. A passing run looks like this:

```
[PASS] starter project structure is present
[PASS] team notes were copied into project/archive
[PASS] old temporary debug note was renamed
```

Score: 3/3 (100%)

Result: terminal setup and basic file operations are ready for the workshop.

If you reach 3/3, your terminal setup and basic file operations are ready for the workshop.

If you finish early. Feel free to start the Week 2 applied session.

Final Tutor Checkpoint

Show your tutor:

- your terminal application or WSL session;
- the path printed by `pwd` inside `week1-rescue`;
- the final checker output showing 3/3;
- the copied notes file in `project/archive`;
- the renamed file in `project/tmp`;
- one sentence explaining the difference between `cp` and `mv`.